Kinvecta

SERIE PARA QUIENES FACILITAN · INMERSIÓN TÉCNICA A

Construye un servidor MCP desde cero

Pocas herramientas, esquemas estrictos, una persona antes de cada acción y tus propios tokens.



Video de 25 minutos · Técnico · DevSecOps · Especificación MCP 2026-07-28

Kinvecta

HOY

Al terminar, las personas participantes podrán…

1

Explicar host, cliente y servidor, y cómo se ve hoy una solicitud

2

Construir un servidor con una herramienta de lectura, una acción que pregunta antes, un recurso y un prompt

3

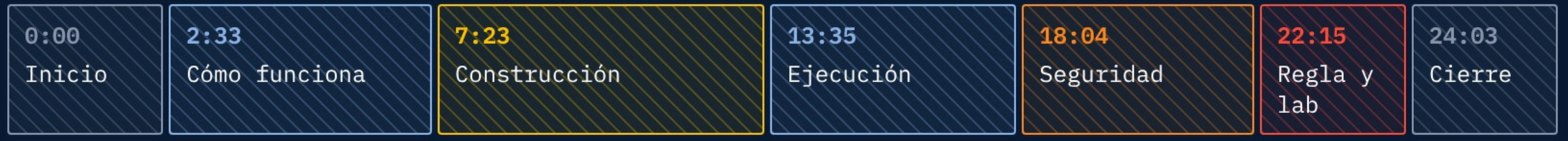
Ejecutarlo por stdio y HTTP, probarlo y conectarlo a un host

4

Asegurarlo: inicio de sesión, sin passthrough, amenazas, controles y publicación

CAPÍTULOS · 25 MIN

a escala



Kinvecta

POR QUÉ CONSTRUIRLO CON CUIDADO

Tres razones para hacerlo bien a la primera

01

Los tutoriales caducaron en julio de 2026

La especificación 2026-07-28 eliminó el handshake y las sesiones; ambos SDKs renombraron sus APIs en v2.

02

Un servidor actúa con tu autoridad

El modelo elige la herramienta, y lo que regresa entra directo al contexto del modelo.

03

Los incidentes son reales

Un servidor npm con puerta trasera copiaba en BCC cada correo; un issue público hizo que un agente filtrara datos privados.

La pregunta de hoy:

¿Cómo construimos un servidor actual y seguro desde el diseño?

LA ARQUITECTURA

Host, cliente, servidor

Host = la app de IA
Ejecuta el modelo, guarda la conversación y pide consentimiento

Cliente = uno por servidor
Un conector dentro del host, en pareja 1:1 con un servidor

Servidor = tu programa
Expone herramientas, recursos y prompts; local o remoto



Kinvecta

EL PANORAMA COMPLETO

Una solicitud, cuatro capas, tres fronteras

El host le pregunta a la persona

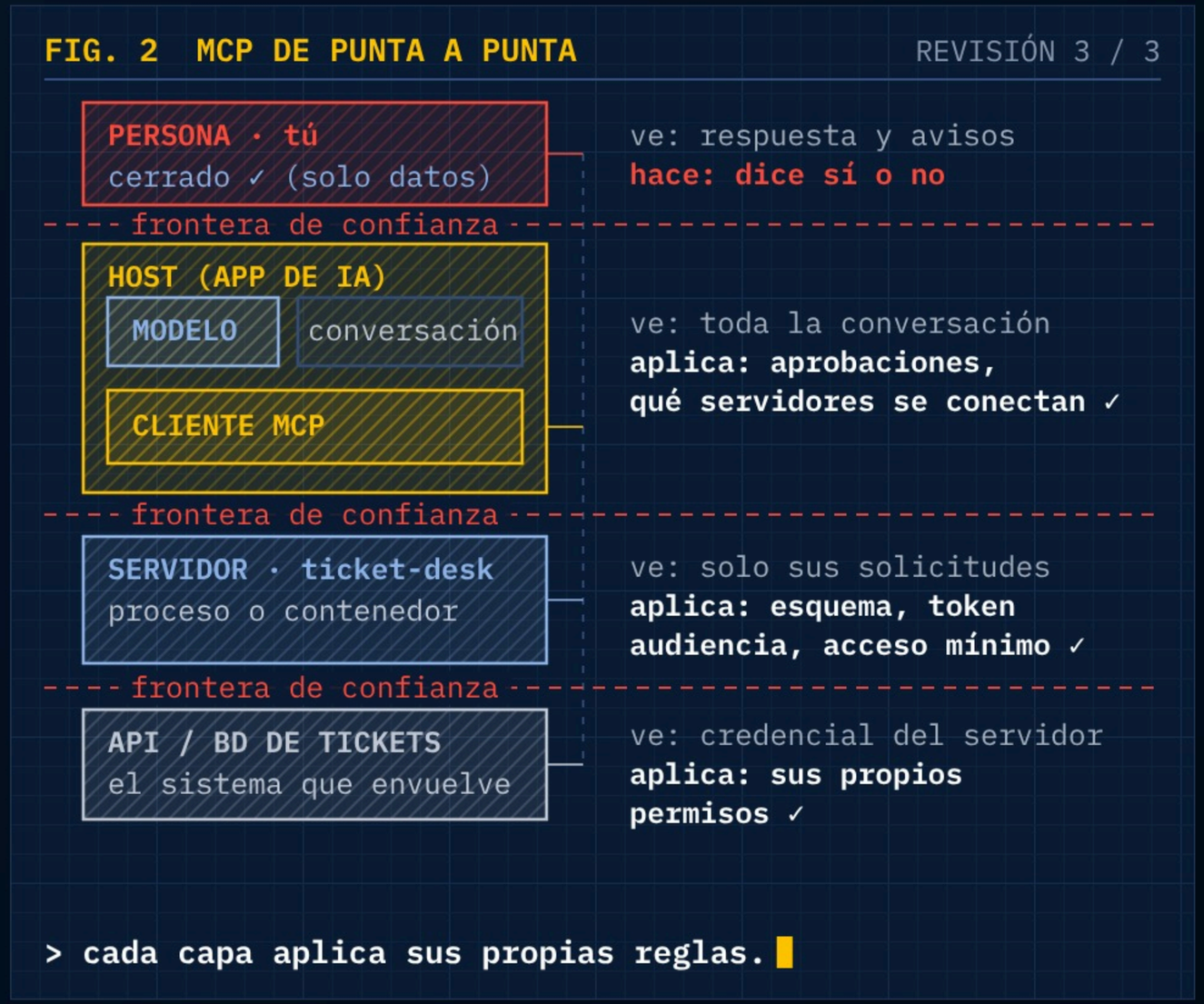
Las aprobaciones y qué servidores se conectan viven en el host

El servidor lo revisa todo

Esquema, audiencia del token y acceso mínimo, en cada llamada

El backend tiene sus reglas

El servidor usa su propia credencial; los permisos del backend siguen aplicando



Kinvecta

PRIMITIVAS DEL SERVIDOR

Tres cosas que un servidor puede ofrecer

Herramientas · decide el modelo

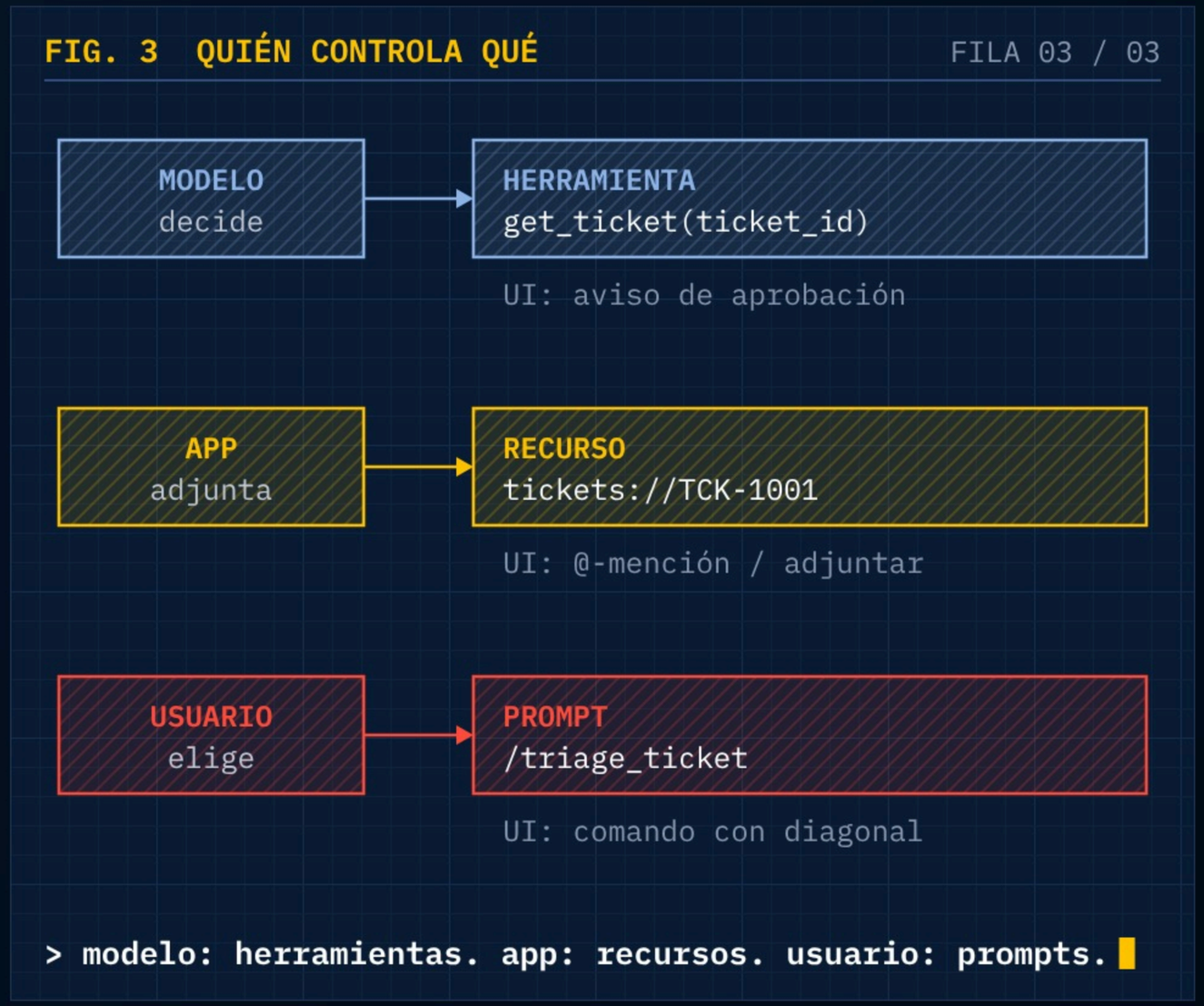
Acciones que el modelo puede pedir, como get_ticket

Recursos · los adjunta la app

Datos de solo lectura por URI, como tickets://TCK-1001

Prompts · los elige el usuario

Plantillas, casi siempre un comando con diagonal, como /triage_ticket



Kinvecta

DEFINICIONES DE HERRAMIENTAS

Cada campo tiene un lector; solo uno aplica

El modelo lee las palabras

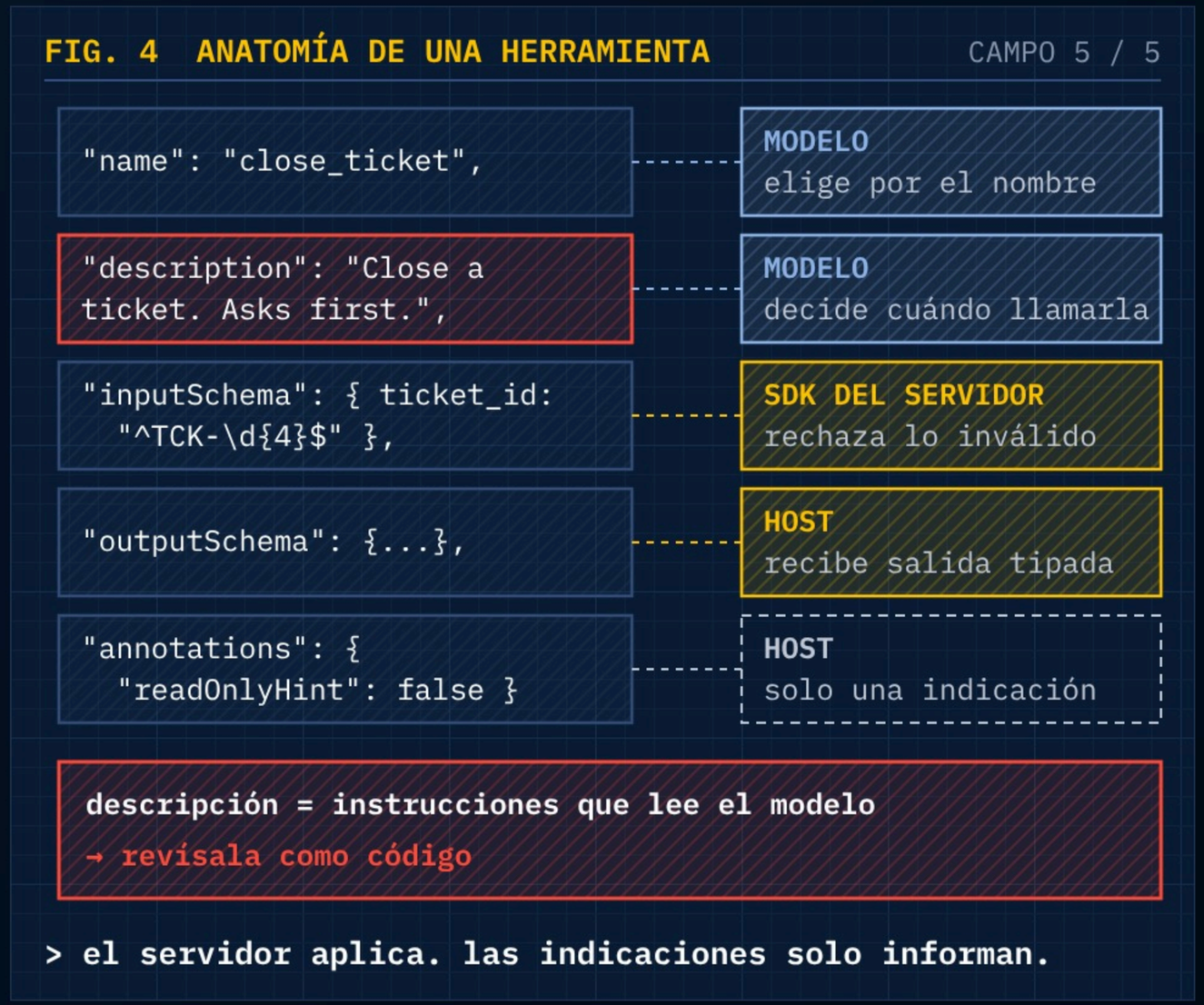
El nombre y la descripción deciden cuándo se llama

El SDK revisa el esquema

La entrada inválida se rechaza antes de que corra tu código

Las anotaciones orientan

Los hosts las usan para la interfaz; aplicar las reglas es tarea del servidor



Kinvecta

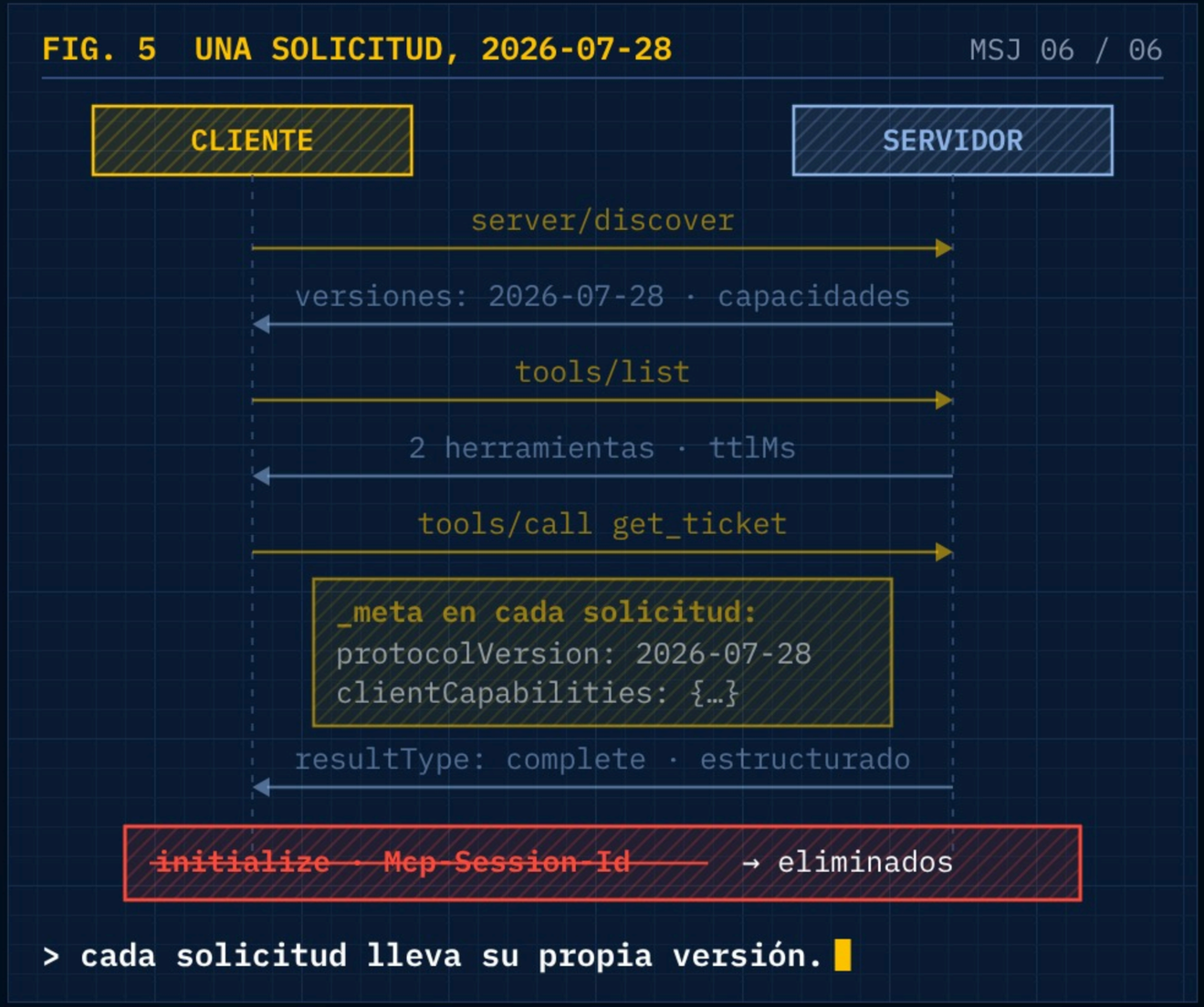
POR DENTRO DEL PROTOCOLO

Cómo se ve hoy una solicitud

JSON-RPC 2.0
Cada mensaje es una solicitud, una respuesta o una notificación

Sin handshake ni sesión
Cada solicitud lleva su propia versión y sus capacidades

server/discover
Los servidores deben implementarlo; los clientes pueden llamarlo primero



Kinvecta

CONSTRUCCIÓN · PREPARACIÓN

Prepara el proyecto con los SDKs actuales

terminal · Python

```
# Python (SDK mcp 2.3.0)
uv init ticket-desk && cd ticket-desk
uv add "mcp[cli]"
uv add --dev pytest inline-snapshot
```

terminal · TypeScript

```
# TypeScript (@modelcontextprotocol/server 2.3.1)
npm init -y && npm pkg set type=module
npm install @modelcontextprotocol/server zod
npm install -D tsx typescript @types/node
```

PY · CAMBIO EN V2

FastMCP → MCPServer

from mcp.server import MCPServer; las opciones de transporte pasan a run()

TS · CAMBIO EN V2

server.tool() → registerTool()

serveStdio(factory), createMcpHandler(factory); solo ESM, Zod v4

INSPECTOR 2.11.0

Requiere Node 22.19 o más reciente

npx @modelcontextprotocol/inspector

Kinvecta

CONSTRUCCIÓN · PYTHON

Una herramienta de solo lectura en 16 líneas

server.py (fragmento)

```python
from mcp.server import MCPServer
from mcp.types import ToolAnnotations

mcp = MCPServer("ticket-desk", version="1.0.0")
TicketId = Annotated[str, Field(pattern=r"^TCK-\d{4}$")]

@mcp.tool(title="Look up a ticket",
          annotations=ToolAnnotations(read_only_hint=True))
def get_ticket(ticket_id: TicketId) -> Ticket:
    """Return one help-desk ticket by ID. Read-only."""
    log.info("get_ticket %s", ticket_id)   # stderr
    if ticket_id not in TICKETS:
        raise ToolError(f"No ticket {ticket_id}.")
    return Ticket(**TICKETS[ticket_id])

mcp.run()   # stdio by default
```

L05 · L09

Las anotaciones de tipo son el esquema

El patrón rechaza “1001; DROP TABLE” antes de que corra tu código

L13

ToolError = un error que lee el modelo

Regresa como isError para que el modelo se corrija

L11

Los logs van a stderr

En stdio, stdout es el canal del protocolo: un print() lo rompe

Kinvecta

CONSTRUCCIÓN · PYTHON

Una acción que primero le pregunta a una persona

server.py (fragmento)

```python
async def confirm_close(ticket_id: str) -> Elicit[Confirm]:
    return Elicit(f"Close ticket {ticket_id}?", Confirm)

@mcp.tool(annotations=ToolAnnotations(read_only_hint=False))
async def close_ticket(
    ticket_id: TicketId,
    resolution: Annotated[str, Field(min_length=5)],
    confirm: Annotated[ElicitationResult[Confirm],
                       Resolve(confirm_close)],
) -> str:
    match confirm:
        case AcceptedElicitation(data=Confirm(confirm=True)):
            TICKETS[ticket_id]["status"] = "closed"
            return f"{ticket_id} closed."
        case _:
            return f"{ticket_id} NOT closed."
```

L08–L09

Resolve oculta confirm

No está en el esquema del modelo, así que el modelo no puede adelantar un “sí”

L15–L16

No o cancelar: nada cambia

Todo lo que no sea un “sí” aceptado regresa NOT closed

TAMBIÉN EN EL ARCHIVO

Un recurso y un prompt

@mcp.resource("tickets://{ticket_id}") y @mcp.prompt()

Kinvecta

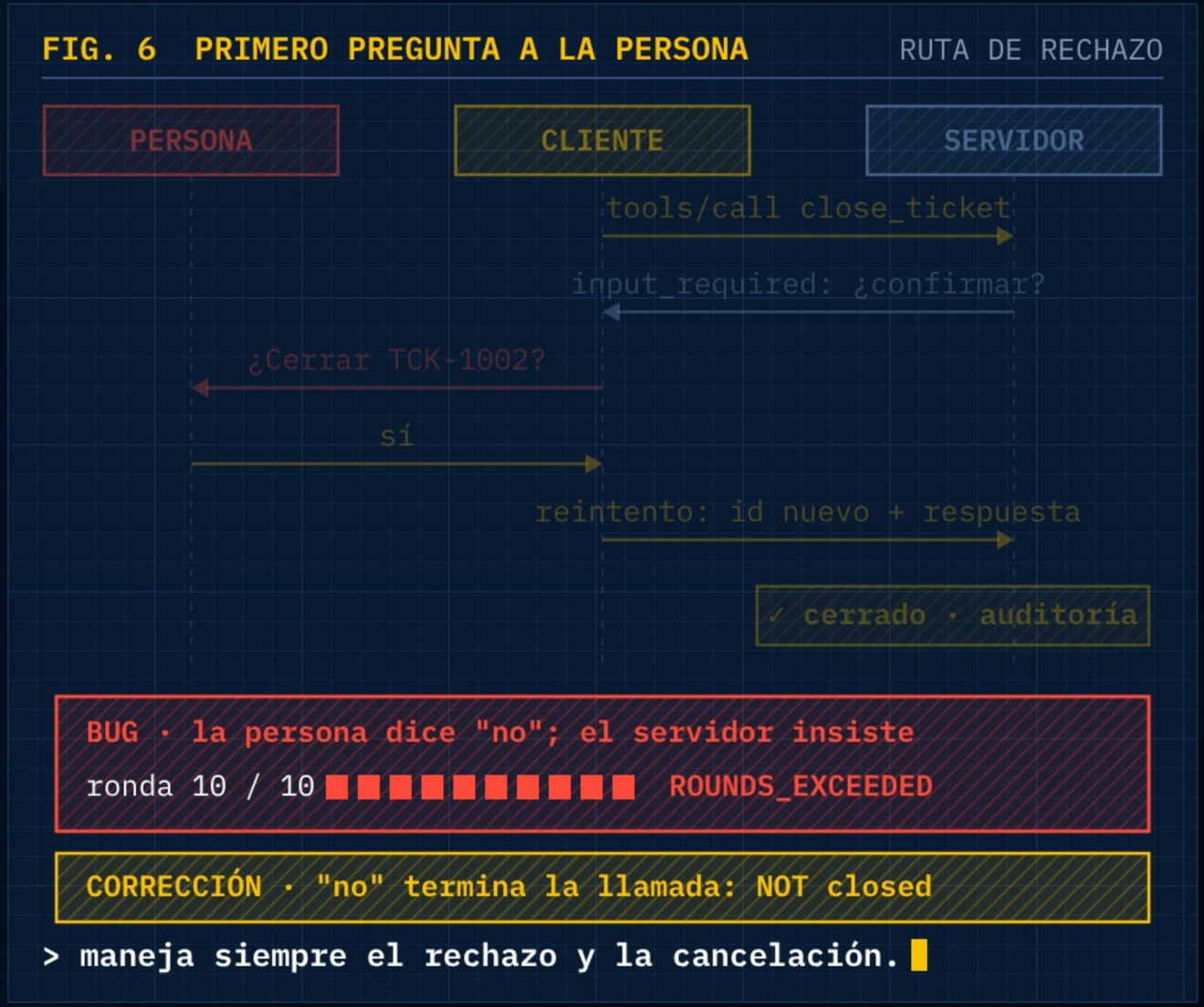
SOLICITUDES DE VARIAS VUELTAS

Cómo le pregunta el servidor a la persona

El servidor da input_required
Nunca le manda su propia solicitud al cliente

El cliente pregunta y reintenta
Con un id de solicitud nuevo y la respuesta de la persona

Maneja el no y el cancelar
Si no, el servidor vuelve a preguntar hasta que el cliente se rinde



Kinvecta

CONSTRUCCIÓN · TYPESCRIPT

La misma acción, con la corrección del rechazo

src/server.ts (fragmento)

```
01  server.registerTool('close_ticket', spec,
02    async ({ ticket_id }, ctx) => {
03      const r = ctx.mcpReq.inputResponses;
04      const view = inputResponse(r, 'confirm');
05      if (view.kind === 'elicit' && view.action !== 'accept')
06        return say(`${ticket_id} NOT closed.`);
07      const answer = acceptedContent(r, 'confirm', Confirm);
08      if (answer === undefined)
09        return inputRequired({ inputRequests: { confirm:
10          inputRequired.elicit({ message: `Close ${ticket_id}?`,
11                                 requestedSchema: Confirm }) } });
12      TICKETS[ticket_id].status = 'closed';
13      console.error(`AUDIT close ${ticket_id}`);  // stderr
14      return say(`${ticket_id} closed.`);
15    });
16  // spec = schema + annotations; say(t) = a text result
```

L04–L06

La corrección: revisa primero no y cancelar

acceptedContent() da undefined si falta, si es no o si se canceló

L08–L11

Sin respuesta aún: regresa input_required

El cliente le pregunta a la persona y reintenta

L13

console.error, nunca console.log

En stdio, stdout es el canal del protocolo

Kinvecta

TRANSPORTES

Dos formas de conectarse

stdio · local

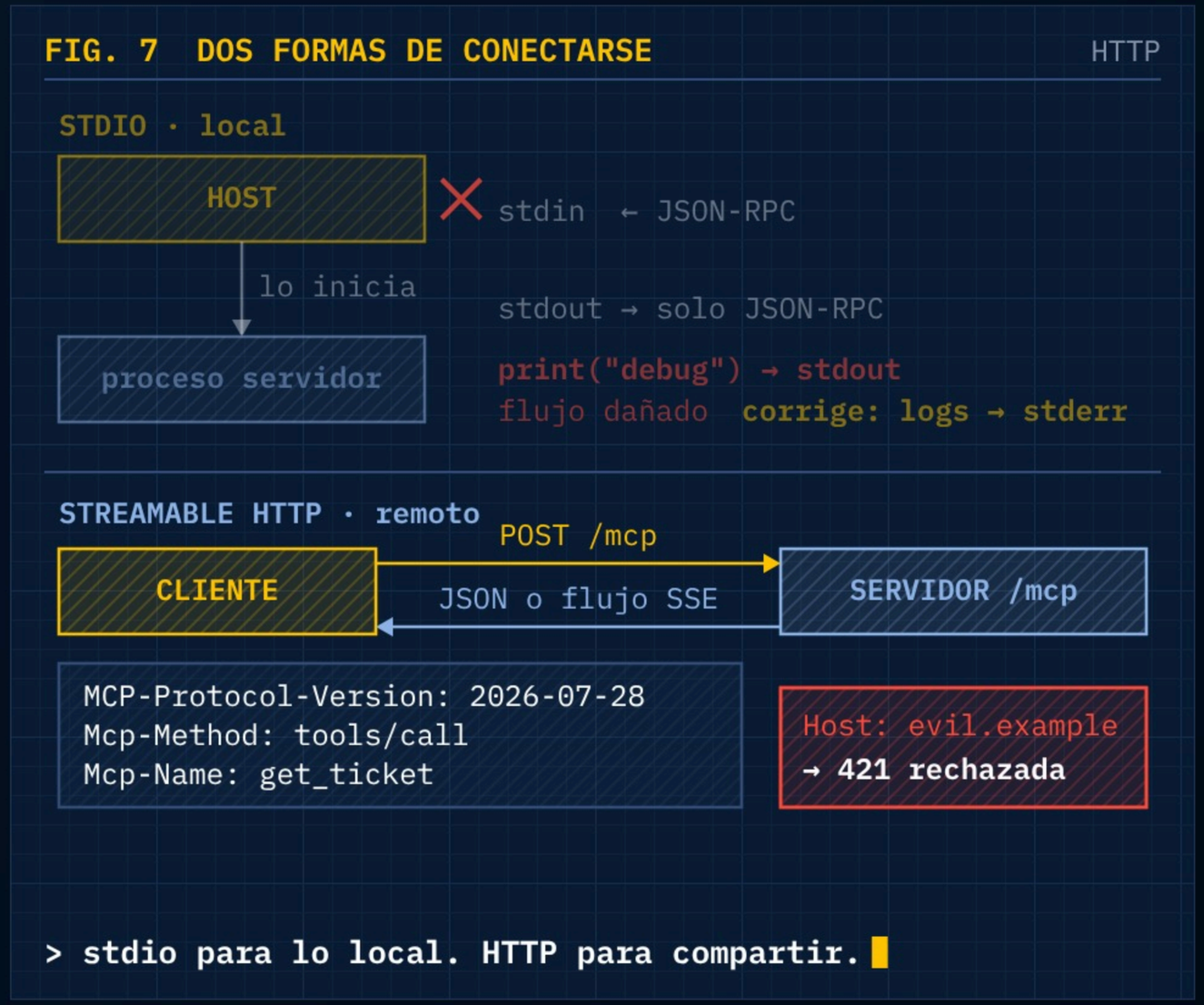
El host inicia tu servidor; por stdout solo van mensajes del protocolo

Streamable HTTP · compartido

Un endpoint POST /mcp; encabezados MCP obligatorios en cada llamada

Cuida la puerta HTTP

En local, escucha en 127.0.0.1; al desplegar, usa una lista de hosts permitidos



Kinvecta

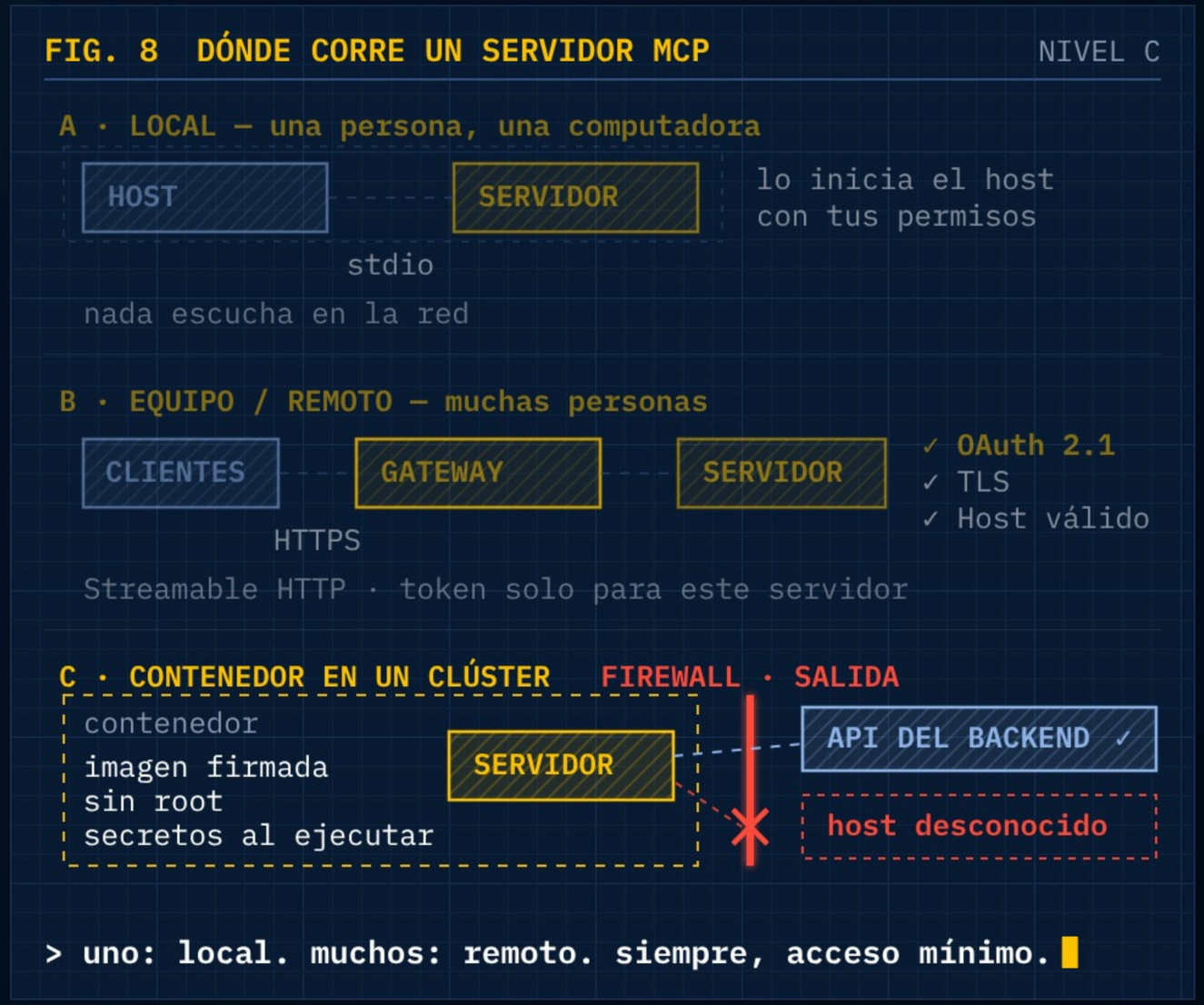
DÓNDE CORRE

Tres lugares para un servidor

Local, para una persona
stdio, con tus permisos, sin puertos abiertos

Remoto, para un equipo
Streamable HTTP tras un gateway; OAuth 2.1, TLS, Host; token solo para este servidor

Contenedor, acceso mínimo
Imagen firmada sin root, secretos al ejecutar, firewall que solo deja pasar la API



Kinvecta

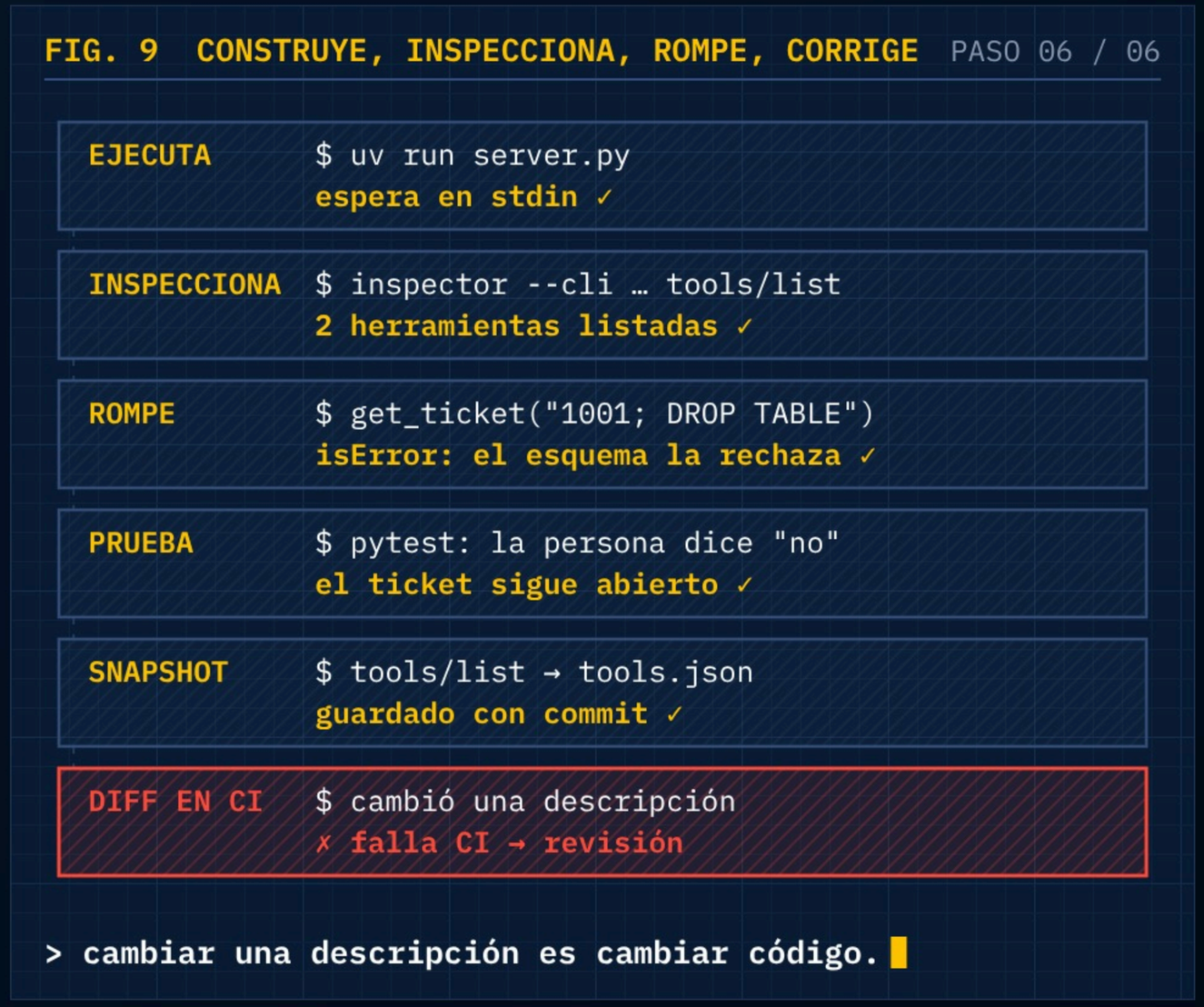
PRUEBAS

Construye, rompe, corrige

MCP Inspector
Modos web, CLI y terminal; prueba las dos eras del protocolo

Pruebas en memoria
Llama al servidor en el mismo proceso: sin puerto ni subproceso

Snapshot de tools/list
Si cambia una descripción, CI falla hasta que alguien la revise



Kinvecta

EJECUCIÓN · CONECTA UN HOST

Conéctalo a un host

claude_desktop_config.json

```
{
  "mcpServers": {
    "ticket-desk": {
      "command": "/ABSOLUTE/PATH/TO/uv",
      "args": ["--directory", "/ABS/ticket-desk",
               "run", "server.py"],
      "env": { "TICKET_API_TOKEN": "…" }
    }
  }
}
```

Claude Code

```
claude mcp add ticket-desk -- \
  uv --directory /abs/ticket-desk run server.py
claude mcp add --transport http ticket-desk \
  https://mcp.example.com/mcp
```

VS Code · .vscode/mcp.json

```
"servers": { "ticket-desk": {
  "type": "stdio", "command": "uv",
  "args": ["run", "server.py"],
  "sandboxEnabled": true } }
```

Kinvecta

INICIO DE SESIÓN REMOTO

OAuth 2.1, y por qué no hay passthrough

Eres el servidor de recursos

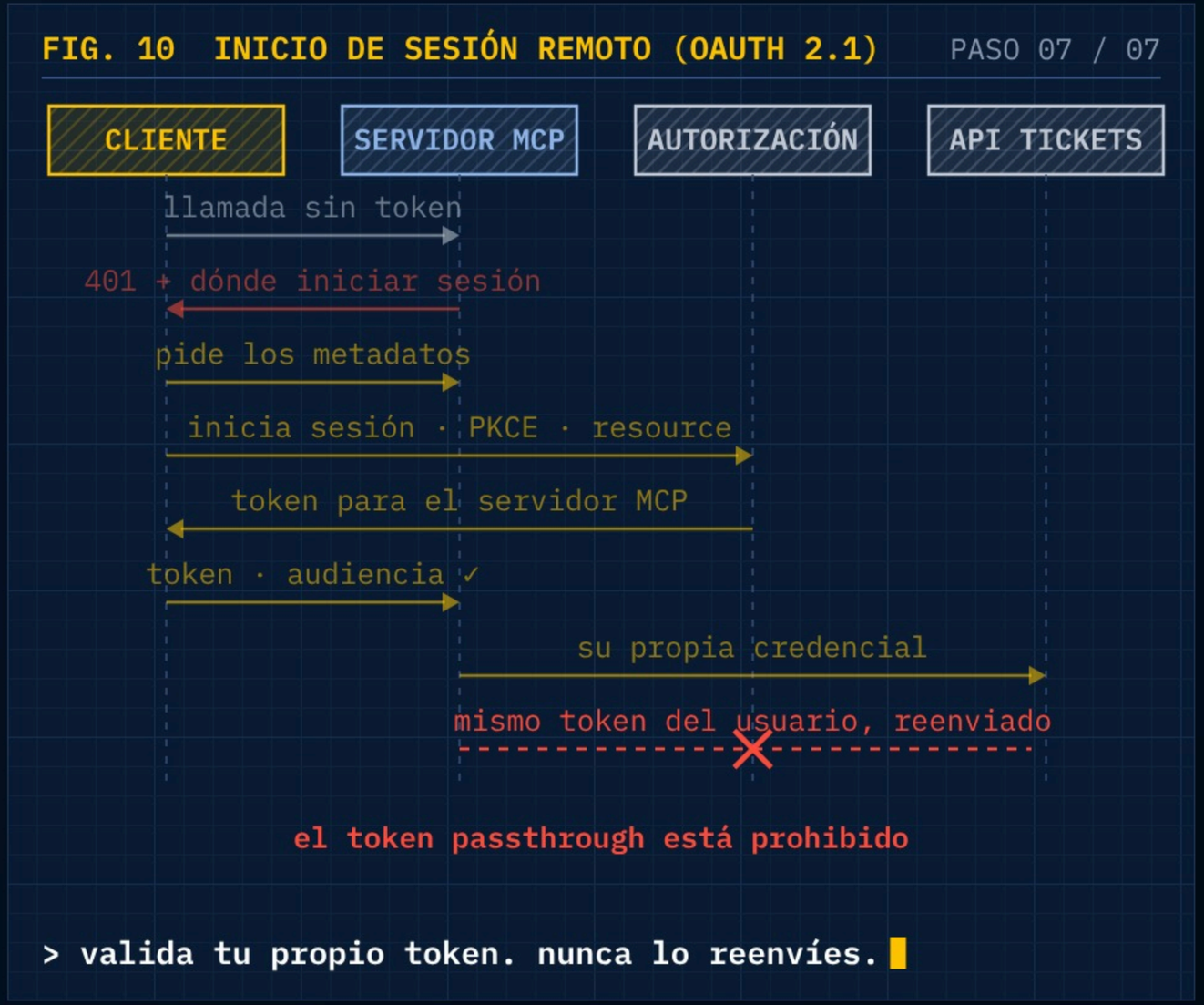
El 401 le indica al cliente dónde iniciar sesión

Tokens solo para este servidor

Revisa la audiencia en cada llamada; 403 si falta el scope

Nunca reenvíes el token

Llama a las APIs de atrás con la credencial propia del servidor



Kinvecta

SEGURIDAD

Cada amenaza tiene un control en el servidor

Anotaciones = indicaciones
readOnlyHint ayuda a la interfaz; quien aplica es tu código

Los incidentes son reales
postmark-mcp, el “toxic agent flow” de GitHub, la fuga de datos en Asana

Relaciónalo con OWASP
OWASP MCP Top 10 (en beta) y las buenas prácticas de la especificación

FIG. 11 AMENAZA → CONTROL — FILA 06 / 06

AMENAZA	CONTROL EN EL SERVIDOR
envenenar herramientas	revisión + snapshot tools/list
rug pull	fija versiones y digests
inyección en resultados	acceso mínimo + confirmación
token passthrough	audiencia + credencial propia
paquete malicioso	fuente verificada + espejo
DNS rebinding	127.0.0.1 + hosts permitidos

> aplícalo en el servidor, no en la indicación.

Kinvecta

DEVSECOPS

Seguridad en cada paso del ciclo

Diseño y construcción

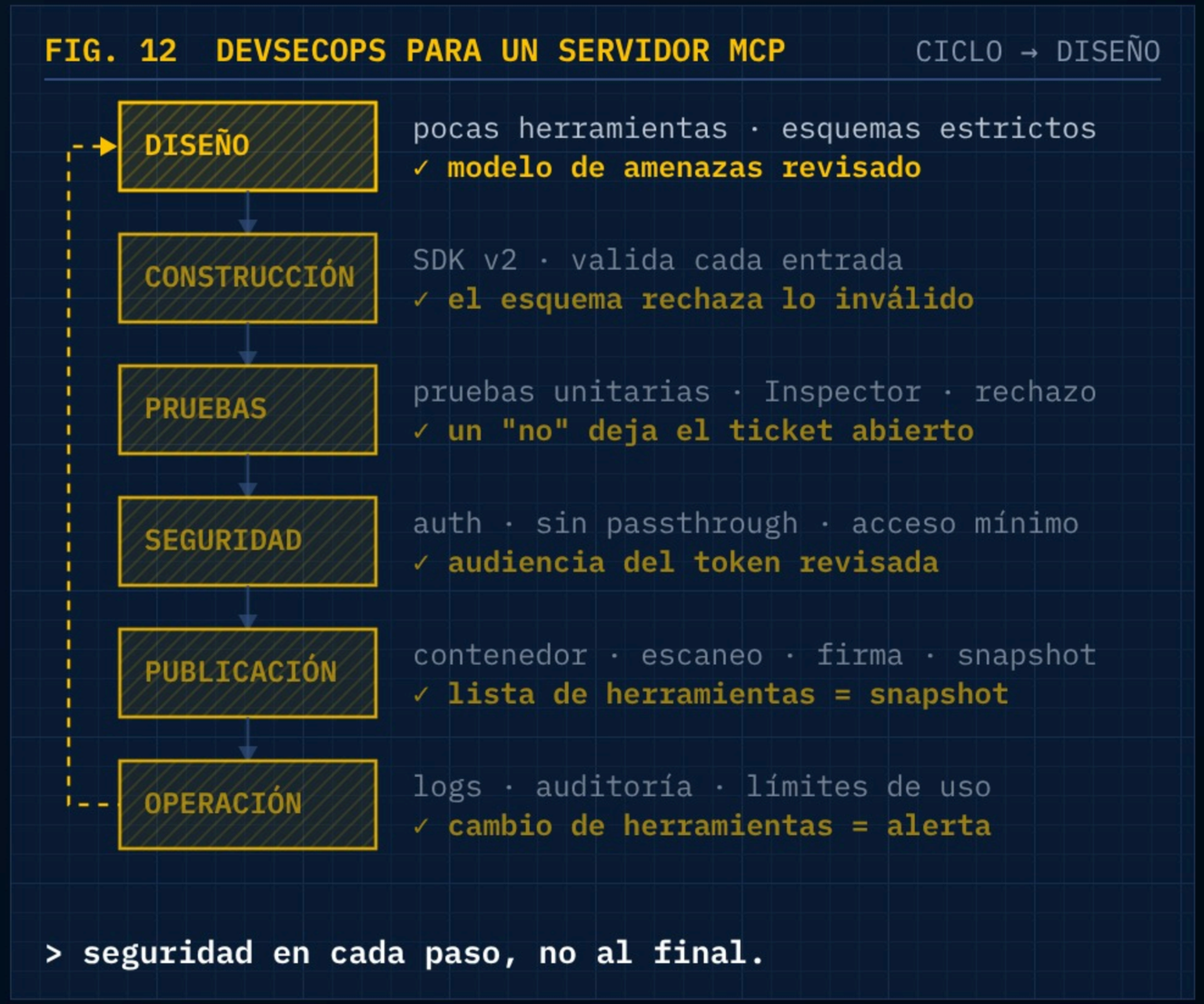
Pocas herramientas, esquemas estrictos, modelo de amenazas, entradas validadas

Pruebas y seguridad

Pruebas, Inspector, ruta de rechazo; audiencia del token revisada, nunca reenviado

Publicación y operación

Contenedor, escaneo, firma, snapshot; logs, auditoría, límites de uso, alertas



Kinvecta

LA REGLA DE ORO

Pocas herramientas, esquemas estrictos.
Una persona confirma cada acción.
Valida tu propio token. Nunca lo reenvíes.

El modelo solo ve tus descripciones y resultados, así que la seguridad se aplica en el servidor.

Kinvecta

LABORATORIO PRÁCTICO

Constrúyelo, inspecciónalo, rómpelo, corrígelo

Pausa y practica

A	Construye	get_ticket y close_ticket en Python o TypeScript, desde el archivo inicial de la carpeta del laboratorio
B	Inspecciona	Inspector CLI: lista las herramientas, llama a TCK-1001 y luego a un ID inválido
C	Rompe	En parejas: envenena una descripción; haz que get_ticket regrese texto inyectado; responde “no” al cierre
D	Corrige	Maneja no y cancelar; guarda el snapshot de tools/list; muestra cómo el diff detecta la descripción envenenada

Kinvecta

PON A PRUEBA LO APRENDIDO

¿Cada participante puede responder esto?

1

¿Cómo sabe un servidor la versión del protocolo del cliente si ya no hay initialize?

2

Tu servidor stdio funciona a mano, pero el host dice “failed”. ¿Por qué?

3

destructiveHint es false. ¿El host puede aprobar la herramienta en automático?

4

¿Por qué un servidor nunca debe reenviar el token que recibió?

5

¿Cómo obtiene un servidor la confirmación de una persona a mitad de una llamada?

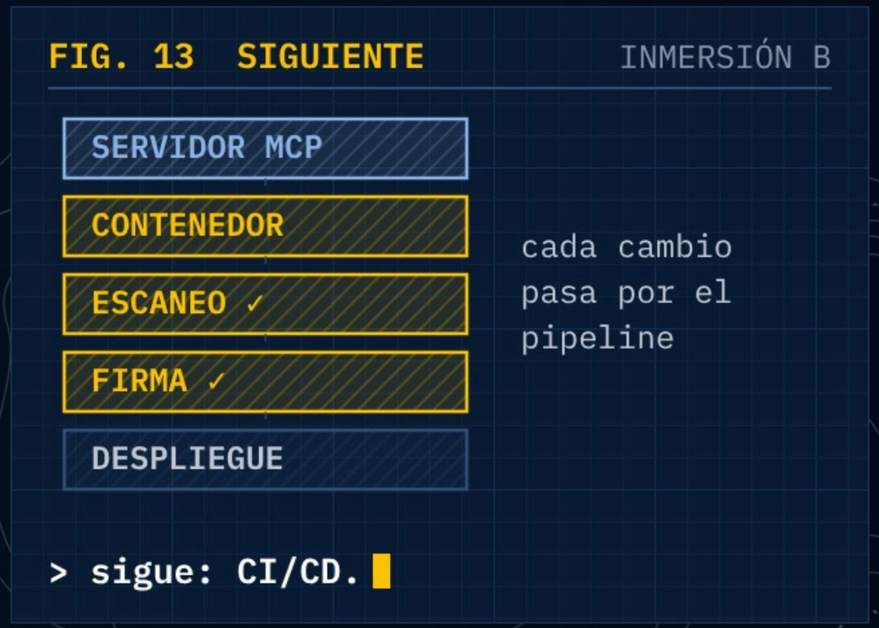
6

Menciona dos controles contra un servidor malicioso o modificado.

Kinvecta

SIGUIENTE · INMERSIÓN TÉCNICA B · CONTENEDORES, ESCANEO Y EL PIPELINE DE CI/CD

¿Cómo llega cada cambio seguro a producción?



Lo que cubre la Inmersión técnica B

- Imágenes y registries
- Herramientas de escaneo
- Firma y SBOMs
- El pipeline de CI/CD